



28 Agosto del 2026

Taller Abstracción de objetos a partir de apps de uso diario

Integrantes

Código estudiantil	Nombre	Usuario Github
	Sergio Elías Pérez Camargo	selpeca
0002620001	Carlos Andres Martinez Jauregui	Andreushin
0182310115	Carlos H. Zarate	DEV-Gordon

APP: Instagram

Instagram es una aplicación de redes sociales enfocada en compartir fotos y videos, que permite a los usuarios publicar contenido, interactuar mediante comentarios y mensajes, y descubrir contenido de otras personas y marcas.

Para qué sirve

- Seguir a otros usuarios y ver su contenido en un feed personalizado
- Publicar y compartir fotos y videos en el perfil personal.
- Subir historias(stories) temporales que desaparecen en 24 horas.
- Publicar videos cortos tipo Reels
- Enviar mensajes directos (DM) y hacer videollamadas.

Características principales

- Funciona con conexión a internet (datos móviles o Wi-Fi).
- Utiliza un nombre de usuario único como identificador de perfil.
- Permite cuentas públicas o privadas controlando quien ve el contenido.
- Incluye filtros y herramientas de edición para fotos y videos.
- Pertenece a la empresa estadounidense Meta

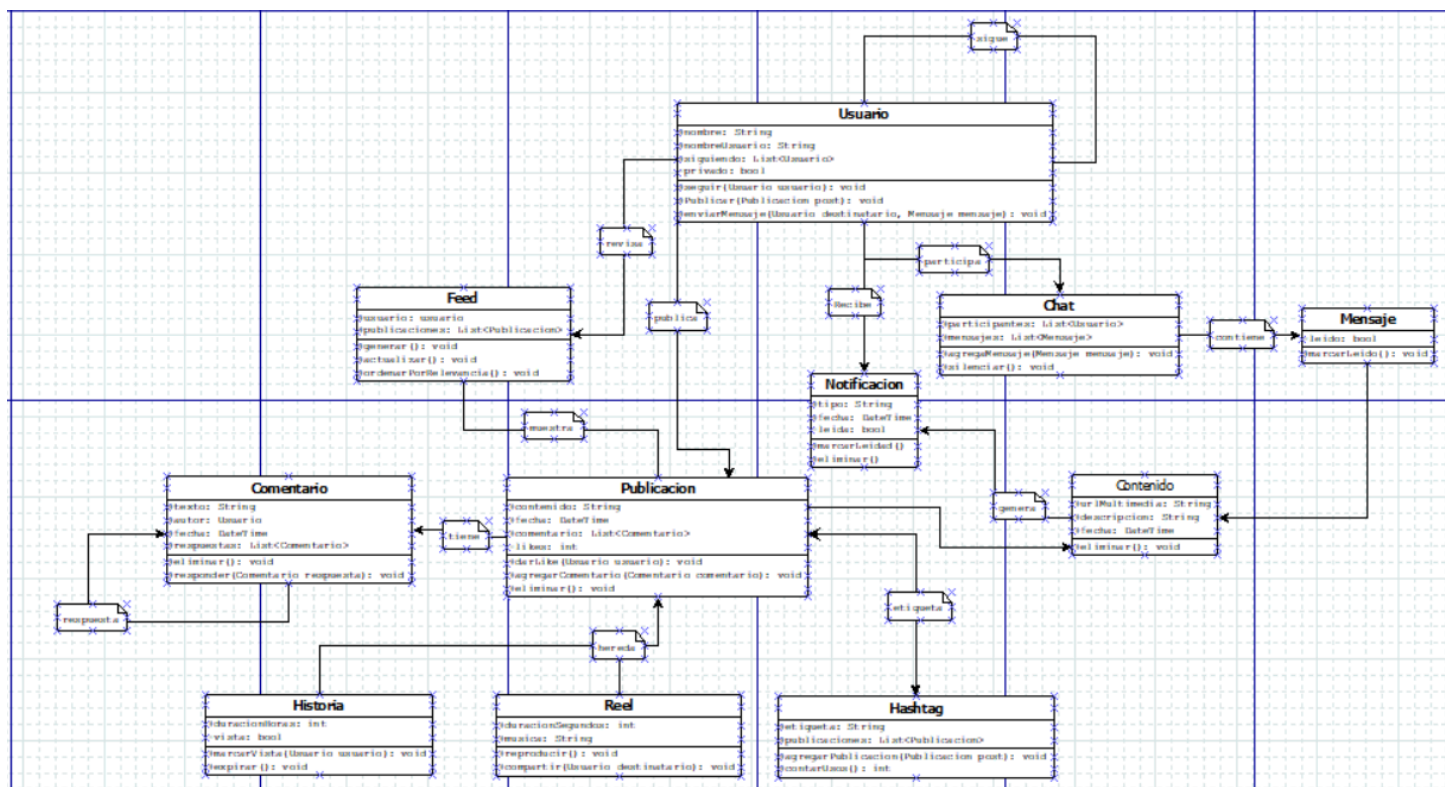


Tabla de clases

Clase	Atributos (nombre: tipo)	Métodos
Usuario	+ nombre: String + nombreUsuario: String + siguiendo: List<Usuario> - privado: bool	+ seguir(Usuario usuario) + publicar(Publicacion post) + enviarMensaje(Usuario destinatario, Mensaje mensaje)
Publicación (hereda de contenido)	+ comentarios : List<Comentario> - likes: int	+ darLike(Usuario usuario) + agregarComentario(Comentario comentario)
Comentario	+ texto: String + autor: Usuario + fecha: DateTime + respuestas: List <Comentario>	+ eliminar() + responder(Comentario respuesta)
Historia (hereda de publicación)	+ duracionHoras: int - vista: bool	+ marcarVista(Usuario usuario) +expirar()
Reel (hereda de publicación)	+ duracionSegundos: int + musica: String	+ reproducir() + compartir(Usuario destinatario)
Mensaje (hereda de contenido)	- leído: bool	+ marcarLeído()
Chat	+ participantes: List<usuario> + mensaje: List<mensaje>	+ agregarMensaje(Mensaje mensaje) + silenciar()
Hashtag	+ etiqueta: String + publicaciones: List<Publicacion>	+ agregarPublicacion(Publicacion post) + contarUsos()
Notificación	+ tipo: String + fecha: DateTime - leída: bool	+ marcarLeída() + eliminar()

feed	+ usuario: Usuario + publicaciones: List<Publicacion>	+ generar() + actualizar() + ordenarPorRelevancia()
Contenido (abstracta)	+ urlMultimedia: String + descripcion: String + fecha: DateTime	+ eliminar()

Diagrama de clases UML





Justificación

Se marcaron como privados los atributos que representan estados internos sensibles, es decir, valores que no deben modificarse libremente desde fuera de la clase sino unicamente a traves de un metodo controlado que garantice la coherencia del sistema:

- `_privado(usuario)`: define si la cuenta es publica o privada. Se mantiene privado porque este valor controla quien puede ver el contenido del usuario; si cualquier parte del sistema pudiera cambiarlo directamente, se rompería la seguridad de la cuenta. Solo debe alterarse mediante un metodo de configuración propio del usuario.
- `_likes(publicacion)`: representa el conteo de reacciones de una publicación. se mantiene privado y solo se modifica a traves de `darLike(Usuario usuario)`, evitando que el contador se incremente o manipule fuera de ese flujo controlado (por ejemplo, dar likes duplicados sin pasar por la validacion del usuario que reacciona).
- `_vista(Historia)`: indica si un usuario ya vio la historia. Es privado porque su unico punto de cambio valido es `marcarVista(Usuario usuario)`; de lo contrario cualquier proceso podria marcar la historia como vista sin que realmente haya sido observada por alguien.
- `_leido(Mensaje)` y `_leida (Notificacion)`: ambos indican si un mensaje o notificacion ya fue revisado por el usuario. Se mantienen privado por la misma razon que en el correspondiente (`marcarLeido()`/ `marcarLeida()`), evitando que el resto del programa lo modifique directamente y asegurando que cada cambio de estado pase por un unico punto controlado.

Se creo contenido una clase abstracta que unifica lo que tiene en comun la clase Publicacion y Mensaje. Es abstracta porque nunca existe un "contenido" por si solo en el sistema siempre se manifiesta como una Publicacion o un Mensaje concretos.

Publicacion hereda de contenido y le suma lo que hace publica(comentarios, likes); mensaje hereda de contenido y le suma que lo hace privada (leido)

Código:

<https://github.com/Andreushin/RiohShots>